

Expense Tracker with Graphical Analysis using Python

1. INTRODUCTION

1.1 Overview

Expense management is an important part of personal finance. Many individuals find it difficult to track their daily expenses manually. This project, **Expense Tracker**, is developed using Python to help users record, store, and analyze their expenses efficiently.

1.2 Purpose of the Project

The purpose of this project is to:

- Track daily expenses
 - Store expense data permanently
 - Analyze expenses using graphs
 - Improve understanding of Python file handling and data visualization
-

2. OBJECTIVES

- To develop a simple expense tracking system using Python
 - To store expense records in an external CSV file
 - To calculate total and monthly expenses
 - To visualize expense data using bar, pie, and line graphs
-

3. Software Requirements

- Operating System: Windows / Linux
 - Programming Language: Python
 - Tools: Jupyter Notebook / VS Code
 - Libraries:
 - datetime
 - matplotlib
-

4. TECHNOLOGIES USED

Technology	Description
Python:	Core programming language
CSV File:	External storage for expenses
Matplotlib:	Graph generation
Jupyter Notebook:	Development environment

5. METHODOLOGY

5.1 Working Principle

1. User enters expense details
2. Program stores data in expenses.csv
3. Data is read from file when required
4. Graphs are generated using matplotlib

◆ 6. MODULE DESCRIPTION

6.1 Add Expense Module

- Accepts category and amount
- Automatically records current date
- Stores data in CSV file

Code:-

```
#Take user input and Save data to CSV file
def add_expense():
    date = datetime.date.today()
    category = input("Enter category: ")
    amount = float(input("Enter amount: "))

    with open(FILE_NAME, "a") as file:
        file.write(f"{date},{category},{amount}\n")

    print("Expense added!")
add_expense()
```

6.2 View Expenses Module

- Reads data from CSV file
- Displays stored expenses

Code:-

```
#Read file and Show stored data
def view_expenses():
    with open(FILE_NAME, "r") as file:
        for line in file:
            print(line.strip())
view_expenses()
```

6.3 Total Expense Calculation Module

- Calculates total spending
- Displays total amount

Code:-

```
#Calculate Total Expense
def total_expense():
    total = 0
    with open(FILE_NAME, "r") as file:
        for line in file:
            total += float(line.strip().split(",")[2])

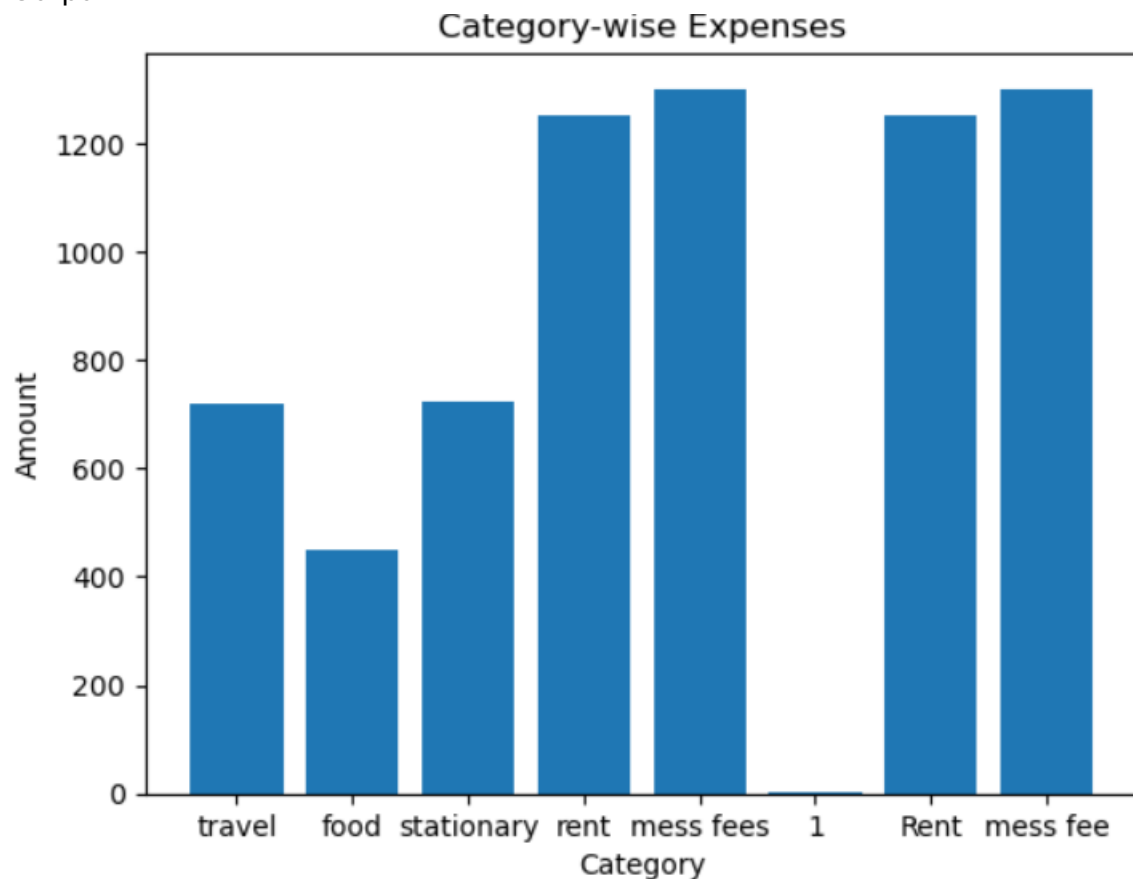
    print("Total Spending:", total)
total_expense()
```

6.4 Graphical Analysis Module

6.4.1 Category-wise Bar Graph

- Shows expense distribution by category

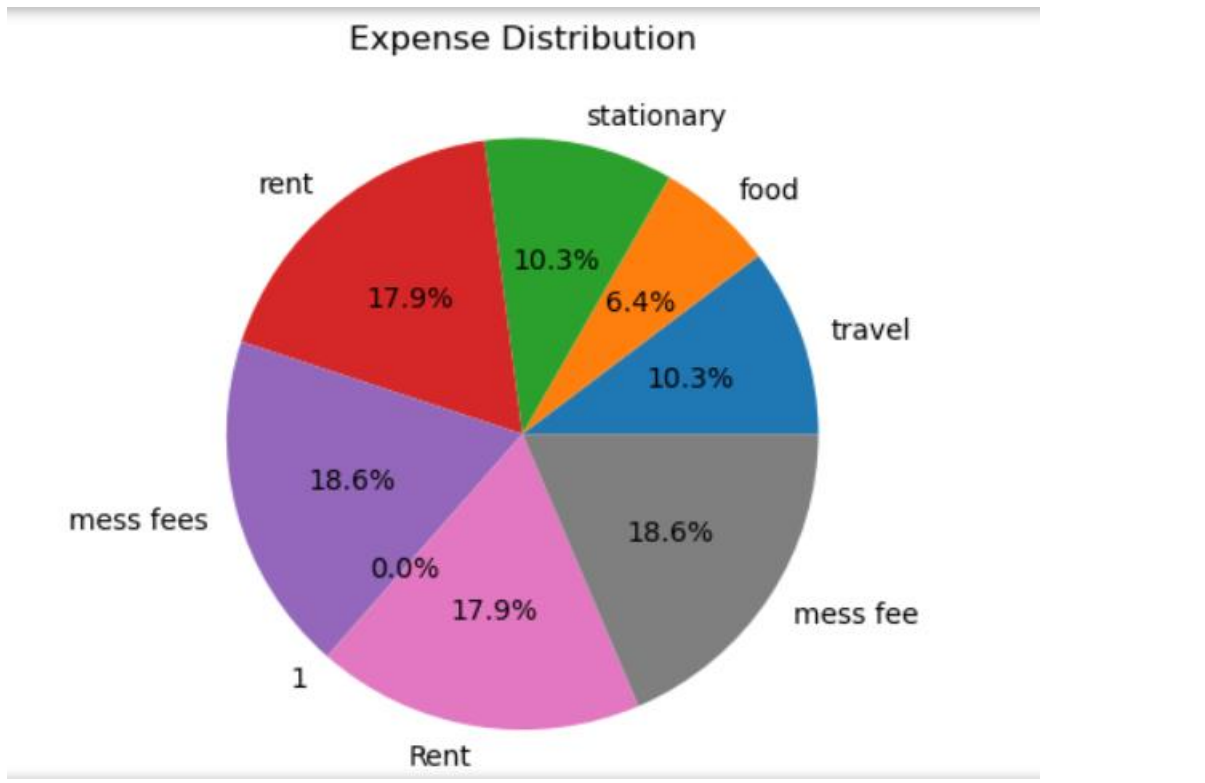
Output:-



6.4.2 Expense Distribution Pie Chart

- Displays percentage share of each category

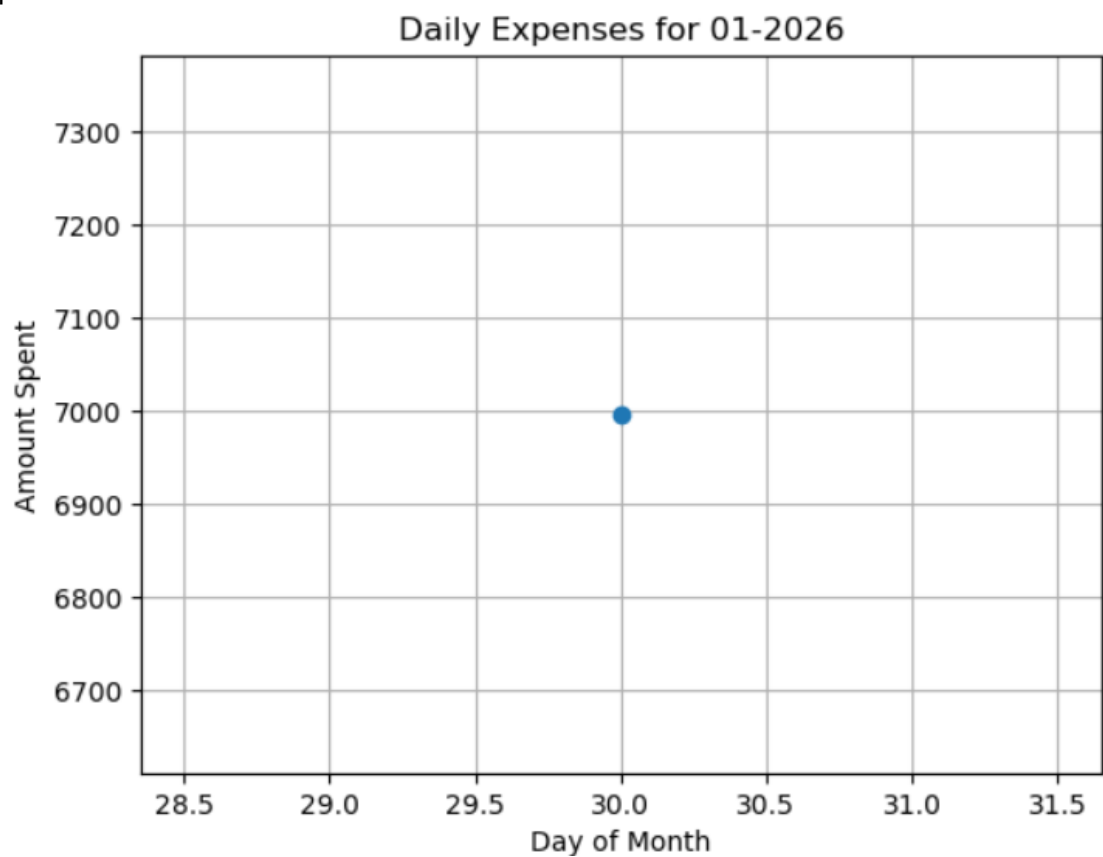
Output:-



6.4.3 Monthly Line Graph

- Shows day-wise expenses for a selected month

Output:-



8. RESULTS AND OUTPUT

- Expenses are stored successfully in CSV file
- Graphs are generated accurately
- User can analyze spending visually

Output:-

```
1. Add Expense
2. View Expenses
3. Total Expense
4. Bar Graph
5. Pie Chart
6. Monthly Line Graph
7. Exit

Enter choice: 1
Enter category: travel
Enter amount: 180
Expense added!
```

```
1. Add Expense
2. View Expenses
3. Total Expense
4. Bar Graph
5. Pie Chart
6. Monthly Line Graph
7. Exit

Enter choice: 2
2026-01-30,travel,120.0
2026-01-30,food,200.0
2026-01-30,stationary,724.0
2026-01-30,rent,1250.0
2026-01-30,mess fees,1300.0
2026-01-30,travel,150.0
2026-01-30,1,2.0
2026-01-30,Rent,1250.0
2026-01-30,mess fee,1300.0
2026-01-30,travel,60.0
2026-01-30,travel,120.0
2026-01-30,food,250.0
2026-01-30,travel,90.0
2026-01-30,travel,180.0
```

```
1. Add Expense
2. View Expenses
3. Total Expense
4. Bar Graph
5. Pie Chart
6. Monthly Line Graph
7. Exit
```

Enter choice: 3

Total Spending: 6996.0

9. ADVANTAGES

- Easy to use
 - Permanent data storage
 - Visual analysis using graphs
 - Suitable for students and beginners
-

10. CONCLUSION

- The Expense Tracker project successfully demonstrates how Python can be used to manage and analyze expenses. The use of external CSV files and graphical visualization makes the system efficient and user-friendly. This project enhances understanding of real-world Python applications.
-

11. REFERENCES

- Python Official Documentation
 - Matplotlib Documentation
 - Class Notes
-